



Ajuste de Parâmetros

Imunologia Computacional

Prof. Ruy Freitas Reis

Pós-Graduação em Modelagem Computacional
Departamento de Ciência da Computação
Universidade Federal de Juiz de Fora



1. Introdução
2. Motivação
3. Medindo Erro
4. Resolvendo o Modelo
5. Evolução Diferencial

Introdução

- Várias fenômenos são modeladas por Equações Diferenciais.
- Um modelo matemático, depois de bem estudado, verificado e validado, pode ser capaz de reproduzir diversos cenários, a depender do conjunto de parâmetros empregado.
- Quando tentamos aplicá-los em diferentes cenários várias dificuldades aparecem, entre elas, quais parâmetros usarmos
- Além disso, muitas vezes temos os resultados experimentais, mas não sabemos qual conjunto de parâmetro melhor reproduz o experimento.



- Nesta aula iremos aprender a como empregar a Evolução Diferencial neste passo tão importante da modelagem computacional
- A evolução diferencial é um método baseado em **população estocástica** que é útil para problemas de otimização global

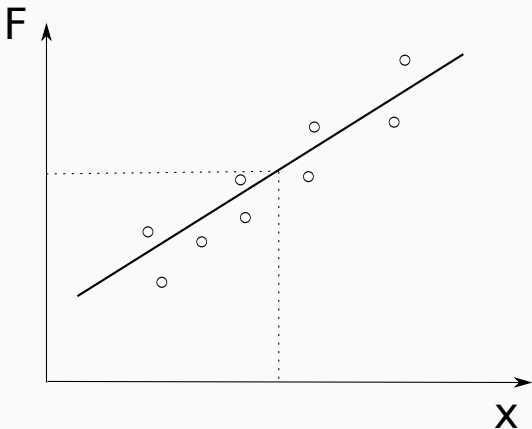
Motivação

- Para ilustrar a ideia, considere agora o problema de **determinar a constante de uma mola**. A Lei de Hooke nos diz que o deslocamento de uma mola é proporcional à força nela aplicada. A questão é como encontrar essa constante de proporcionalidade a partir de dados experimentais.

$$\text{Lei de Hooke} \Rightarrow F = kx$$

- Suponha que possamos realizar vários experimentos medindo forças aplicadas à mola e os seus respectivos deslocamentos.

O problema consiste em encontrar uma reta que melhor aproxime esses dados. A inclinação da reta irá nos fornecer a constante k da mola.



- Vamos chamar de $f(x)$ a função que queremos aproximar por uma outra função $g(x)$.
- Nem sempre temos uma expressão analítica para $f(x)$.
- Iremos partir da hipótese de que temos algumas informações sobre a forma de $g(x)$. Poderíamos saber, através da observação dos dados, por exemplo, que $g(x)$ é uma reta,

$$g(x) = c_0 + c_1x$$

ou que é uma parábola

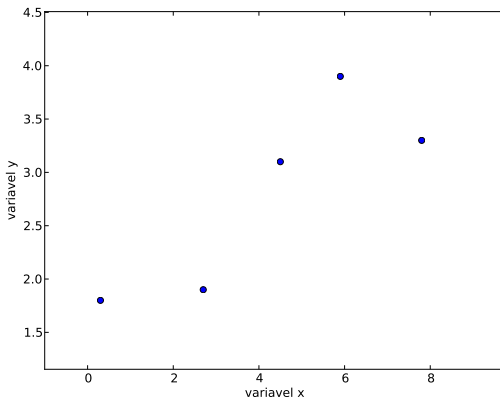
$$g(x) = c_0 + c_1x + c_2x^2$$

ou que tenha alguma outra forma específica.

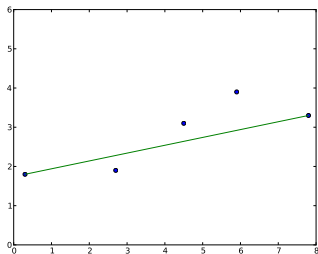
- Suponha que temos a seguinte tabela de dados

x_i	0.3	2.7	4.5	5.9	7.8
y_i	1.8	1.9	3.1	3.9	3.3

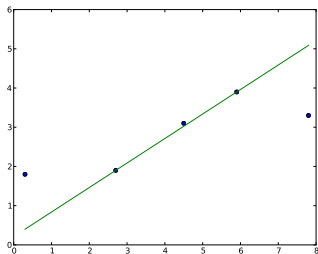
- Vamos analisar os dados.



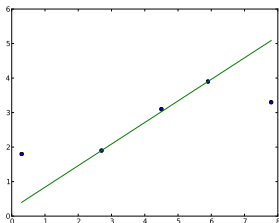
- Podemos considerar que existe uma relação aproximadamente linear entre as variáveis.
- Logo, podemos desprezar alguns pontos da tabela, escolher 2 pontos e usar polinômios interpoladores lineares para aproximação. Ou seja $g(x) = c_0 + c_1x$.
- Para $(0.3, 1.8)$ e $(7.8, 3.3)$ temos $g_1(x) = 1.74 + 0.2x$.



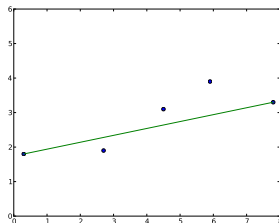
- Podemos considerar que existe uma relação aproximadamente linear entre as variáveis.
- Logo, podemos desprezar alguns pontos da tabela, escolher 2 pontos e usar polinômios interpoladores lineares para aproximação. Ou seja $g(x) = c_0 + c_1x$.
- Para $(2.7, 1.9)$ e $(5.9, 3.9)$ temos $g_2(x) = 0.2125 + 0.625x$.



- Qual aproximação é melhor? $g_1(x)$ ou $g_2(x)$?
- Como verificar a qualidade da aproximação?

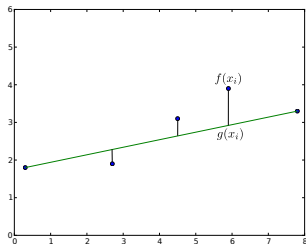


(a) $g_1(x) = 1.74 + 0.2x$



(b) $g_2(x) = 0.2125 + 0.625x$

- Para obter as aproximações g_1 e g_2 desprezamos vários dados da tabela para fazer a interpolação, o que não é muito conveniente de se fazer.
- Um modo de se verificar a qualidade da aproximação é calculando a soma de todas as distâncias verticais de $f(x_i)$ e $g(x_i)$ ao quadrado, isto é



$$\begin{aligned} E^2 &= \sum_{i=1}^m r(x_i)^2 = \sum_{i=1}^m [f(x_i) - g(x_i)]^2 \\ &= \sum_{i=1}^m [f(x_i) - c_0 - c_1 x_i]^2 \end{aligned}$$

Para $g_1(x) = 1.74 + 0.2x$

i	x_i	$f(x_i)$	$g(x_i)$	r_i^2
1	0.3	1.8	1.8	0
2	2.7	1.9	2.78	0.144
3	4.5	3.1	2.64	0.2116
4	5.9	3.9	2.94	0.9604
5	7.8	3.3	3.3	0

$$E^2 = 1.316$$

Para $g_2(x) = 0.2125 + 0.625x$

i	x_i	$f(x_i)$	$g(x_i)$	r_i^2
1	0.3	1.8	0.4	1.96
2	2.7	1.9	1.9	0
3	4.5	3.1	3.025	0.0056
4	5.9	3.9	3.9	0
5	7.8	3.3	2.275	1.051

$$E^2 = 3.0166$$

- Para $g_1(x)$ temos $E^2 = 1.316$
- Para $g_2(x)$ temos $E^2 = 3.0166$
- Logo, $g_1(x)$ é mais adequado de acordo com esse critério de erros ao quadrado.
- **Como escolher c_0 e c_1 de forma que E^2 seja mínimo?**

- Uma tentativa simplista para descobrir isso seria uma busca exaustiva (força bruta)
- Mas a função que desejamos ajustar possui um número alto de parâmetros.
- Outra alternativa é fazer esse ajuste de forma direta (O método dos mínimos quadrados, por exemplo)
- Mas mas isso pode ser dificultado a depender do tipo de função que desejamos ajustar (não linearidade, por exemplo)

Medindo Erro

Medindo Erro

Teoria

Uma forma bem conhecida de medir erros são através de normas.
As normas mais conhecidas são norma-1 e norma-2:

$$\|e\|_1 = \sum_{i=1}^N |e_i|$$
$$\|e\|_2 = \left(\sum_{i=1}^N |e_i|^2 \right)^{1/2}.$$

onde N é o tamanho do vetor, ou número de pontos que temos, e é o vetor de resíduos. Neste caso, $e = f - g$, onde f são os dados que desejamos ajustar a uma função g

Similarmente todas as outras normas podem ser derivadas do caso geral, norma- q , definida por:

$$\|e\|_q = \left(\sum_{i=1}^N |e_i|^q \right)^{1/q}.$$

Quando $q \rightarrow \infty$, temos a norma-inf

$$\|e\|_\infty = \max_{1 \leq i \leq N} |e_i|.$$

Quando estamos lidando com dados onde a magnitude dos erros é grande, muitas vezes é válido lidar com uma abordagem relativa do erro. Desta forma o erro relativo pode ser calculado da seguinte forma:

$$\begin{aligned} R_E(f, g) &= \frac{\|f - g\|}{\|g\|} \\ &= \frac{\|e\|}{\|g\|}. \end{aligned}$$

O erro relativo pode ser calculado utilizando qualquer uma das normas vistas anteriormente.

Medindo Erro

Prática em Python

Isto pode ser facilmente feito em Python da seguinte forma.
Primeiro precisaremos da biblioteca `numpy`:

```
1 import numpy as np
```

Vamos definir os vetores **f**, **g** e resíduo **e**:

```
1 f = np.array([1.0, 2.0, 3.0, 4.0, 5.0])  
2 g = np.array([1.1, 2.1, 2.9, 4.5, 5.6])  
3  
4 e = f - g
```

Por fim, podemos utiliza as normas implementadas na biblioteca numpy e imprimir o resultado na tela

```
1 norma_1 = np.linalg.norm(e, 1)
2 norma_2 = np.linalg.norm(e, 2)
3 norma_inf = np.linalg.norm(e, np.inf)
4
5 print("norma_1 = ",norma_1)
6 print("norma_2 = ",norma_2)
7 print("norma_inf = ",norma_inf)
```

Neste caso, obteremos:

```
norma_1 = 1.4
norma_2 = 0.7999999999999998
norma_inf = 0.5999999999999996
```

O calculo do erro relativo poderia ser uma continuação da implementação anterior

```
1 erro_rel_1 = norma_1/np.linalg.norm(g, 1)
2 erro_rel_2 = norma_2/np.linalg.norm(g, 2)
3 erro_rel_inf = norma_inf/np.linalg.norm(g, np.inf)
4
5 print("Erro Relativo com norma_1 = ",erro_rel_1)
6 print("Erro Relativo com norma_2 = ",erro_rel_2)
7 print("Erro Relativo com norma_inf = ",erro_rel_inf)
```

Neste caso, obteremos:

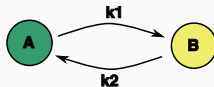
```
Erro Relativo com norma_1 = 0.08641975308641975
Erro Relativo com norma_2 = 0.09874285972226246
Erro Relativo com norma_inf = 0.10714285714285708
```

Resolvendo o Modelo

- Uma biblioteca simples de ser utilizada para resolver um sistema de equações diferenciais ordinárias é o ODEInt do SciPy.
- Ela resolve o sistema de EDOs utilizando o lsoda da biblioteca odepack desenvolvida em FORTRAN
- Resolve o problema do valor inicial para sistemas de EDOs stiff ou non-stiff de primeira ordem

Resolva o seguinte sistema de EDOs que simula uma isomerização:

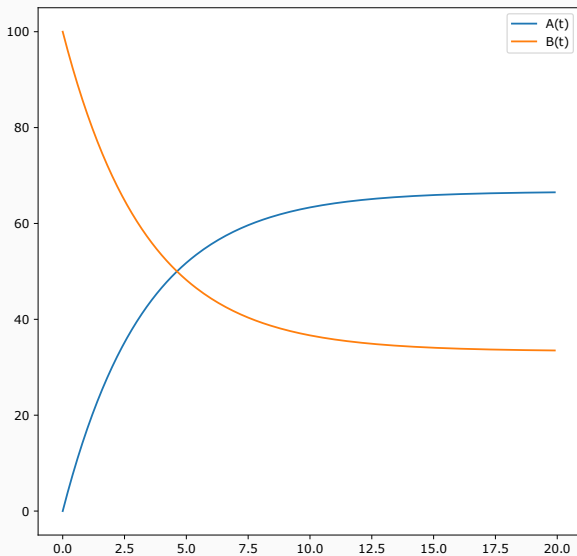
$$\begin{cases} \frac{d[A]}{dt} = -k_1[A] + k_2[B] \\ \frac{d[B]}{dt} = k_1[A] - k_2[B] \end{cases}$$



Considere $t \in [0, 20]$, $A_0 = 0.0$, $B_0 = 100.0$, $k_1 = 0.1$, $k_2 = 0.2$.

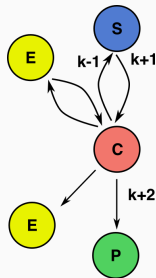
```
1 from scipy.integrate import odeint
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 def model_isomerizacao(y, t, k1, k2):
6     A, B = y[0], y[1]
7
8     dAdt = -k1*A + k2*B
9     dBdt = k1*A - k2*B
10
11     return [dAdt, dBdt]
12
13 t0=0.0; #intervalo de tempo
14 tf=20.0;
15 a0=0; #condicao inicial
16 b0=100;
17 k1 = 0.1 #k+
18 k2 = 0.2 #k-
19 t = np.arange(t0,tf,0.1)
20 P0= [a0,b0]
21 mmargs = (k1,k2)
22 Y = odeint(model_isomerizacao,P0,t, args=(mmargs))
23 plt.plot(t,Y[:,0],label="A(t)")
24 plt.plot(t,Y[:,1],label="B(t)")
25 plt.legend(loc='upper right')
26 plt.show()
```

Exemplo Resolvido



1. Simule uma reação enzimática pelo modelo de Michaelis-Menten

$$\begin{cases} \frac{d[S]}{dt} = k_{-1}[C] - k_{+1}[S][E] \\ \frac{d[E]}{dt} = (k_{-1} + k_{+2})[C] - k_{+1}[S][E] \\ \frac{d[C]}{dt} = -(k_{-1} + k_{+2})[C] + k_{+1}[S][E] \\ \frac{d[P]}{dt} = k_{+2}[C] \end{cases}$$



Utilize as condições iniciais: $s_0 = 10.0$, $e_0 = 2.0$ e $c_0 = p_0 = 0.0$; e parâmetros $k_{1+} = 0.01$, $k_{1-} = 0.02$ e $k_{2+} = 0.03$. Considere $t \in [0, 1000]$

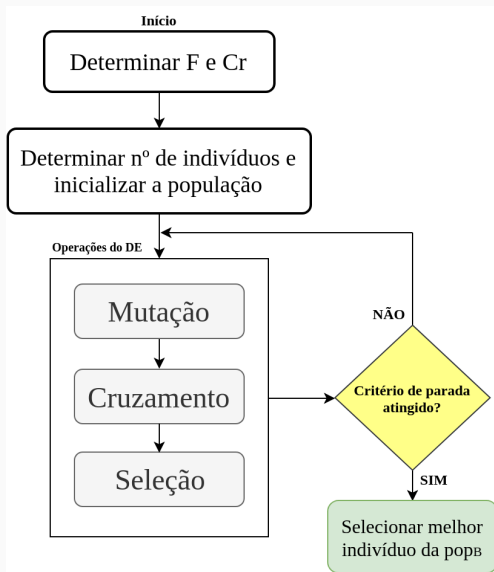
2. O modelo SIRD diferencia Suscetível, Infectado, Recuperado (significando especificamente os indivíduos que sobreviveram à doença e agora estão imunes) e Mortos, respectivamente. Este modelo usa o seguinte sistema de equações diferenciais

$$\begin{aligned}\frac{dS}{dt} &= -\frac{\beta IS}{N}, \\ \frac{dI}{dt} &= \frac{\beta IS}{N} - \gamma I - \mu I, \\ \frac{dR}{dt} &= \gamma I, \\ \frac{dD}{dt} &= \mu I,\end{aligned}$$

onde β, γ, μ são as taxas de infecção recuperação e mortalidade, respectivamente. Resolva o modelo SIRD considerando $t \in [0, 100]$, $S_0 = 997$, $I_0 = 3$, $R_0 = D_0 = 0$,

Evolução Diferencial

- A Evolução diferencial é um algoritmo de otimização simples e eficiente que foi proposto por Rainer Storn e Kenneth Price em 1997.
- Trata-se de um método estocástico de busca
- O DE é um método, que além de ser de fácil implementação, tem ótimo desempenho numa grande classe de problemas
- O DE pode ser descrito como uma manipulação de indivíduos que representam as soluções candidatas.
- No decorrer das gerações, essas soluções candidatas sofrem modificações de mutação e cruzamento, onde são geradas novas soluções candidatas, e logo após é feita a seleção e o ciclo se repete.



Evolução Diferencial

Ajuste de parâmetros

Para ajustar uma EDO ou sistema de EDOs utilizando a DE precisamos:

- 1) Definição e resolução do modelo dado um conjunto de parâmetros e condições iniciais
- 2) Conjunto de parâmetros e/ou condições iniciais
- 3) Espaço de busca para cada valor a ser ajustado, *i.e.* limites superior e inferior dos valores
- 4) Função objetivo para ser minimizada

Uma ideia simples que pode usada no ajuste é: **minimizar a diferença entre as curvas do modelo para os dados fornecidos (erro relativo)** de acordo com seguinte equação:

$$\min_p \left(\omega_1 R_E(C_1, \hat{C}_1) + \omega_2 R_E(C_2, \hat{C}_2) + \cdots + \omega_i R_E(C_i, \hat{C}_i), \right),$$

onde C_i é a i -ésima curva do modelo a ser ajustada, $\hat{C}_i$ é os dados experimentais em relação a curva C_i , ω_i o peso dado ao ajuste da curva C_i . Por fim R_E é o erro relativo:

$$R_E(\lambda, \hat{\lambda}) = \frac{\|\lambda(t, p) - \hat{\lambda}(t)\|}{\|\hat{\lambda}(t)\|},$$

Evolução Diferencial

Prática em Python

Isto pode ser facilmente feito em Python da seguinte forma.
Primeiro precisaremos das bibliotecas `numpy`, `matplotlib.pyplot`
e a função `differential_evolution` da biblioteca
`scipy.optimize`:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import differential_evolution
```

Vamos definir a função que queremos ajustar

```
1 def model_reta(x,c0,c1):  
2     return c0+c1*x
```

e a função objetivo e os dados que iremos utilizar no ajuste, seguindo a ideia explicada anteriormente:

```
1 x = np.array([0.3, 2.7, 4.5, 5.9, 7.8])  
2 y = np.array([1.8, 1.9, 3.1, 3.9, 3.3])  
3  
4 def model(params):  
5     '''  
6     Descricao dos parametros:  
7  
8     params[0]=> c0  
9     params[1]=> c1  
10    '''  
11    sol = model_reta(x,params[0], params[1])  
12    #utilizou norma-2, mas qualquer uma poderia ser utilizada  
13    erro = np.linalg.norm(y-sol, 2)/np.linalg.norm(y, 2)  
14  
15    return [erro, sol, x]
```

Agora vamos utilizar a função `differential_evolution` da biblioteca `scipy.optimize`, para isto precisamos de uma função objetivo que o seu único retorno seja o erro. Desta forma utilizaremos esse 'truque'

```
1 def model_adj(x):  
2     result = model(x)  
3     return result[0]
```

Por fim, vamos definir o espaço da busca do ajuste e chamar a função

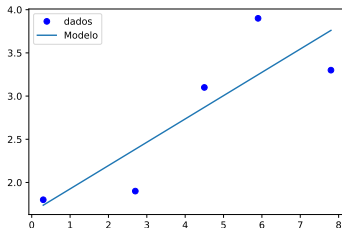
```
1 bounds = [(0, 2), (0, 2)]  
2 result = differential_evolution(model_adj, bounds, strategy='best1bin', disp=  
    True)  
3 print(result.x)
```

A saída da função é a seguinte

```
differential_evolution step 1: f(x)= 0.157204
differential_evolution step 2: f(x)= 0.157204
differential_evolution step 3: f(x)= 0.157204
differential_evolution step 4: f(x)= 0.1491
differential_evolution step 5: f(x)= 0.1491
differential_evolution step 6: f(x)= 0.149082
differential_evolution step 7: f(x)= 0.149082
differential_evolution step 8: f(x)= 0.148184
differential_evolution step 9: f(x)= 0.147945
differential_evolution step 10: f(x)= 0.147734
differential_evolution step 11: f(x)= 0.147734
[1.65594158 0.2698251 ]
```

Para visualizar o resultados do ajuste, podemos utilizar o `matplotlib.pyplot` da seguinte forma

```
1 erro, sol, x = model(result.x)
2
3 plt.plot(x, y, 'bo', label='dados');
4 plt.plot(x, sol, label='Modelo');
5 plt.legend()
6 plt.show()
```



- 1) Utilizando o Modelo SIRD simule a pandemia de COVID-19 utilizando dados de algum país/estado a sua escolha. Use os dados disponibilizados pelo *COVID-19 Data Repository* do *Center for Systems Science and Engineering (CSSE)* em *Johns Hopkins University* do hospital <http://github.com/CSSEGISandData/COVID-19>. Para fazer o ajuste de parâmetros utilize poucos dias de simulação (50 a 100 dias), pois este modelo SIRD é bem simples para conseguir simular toda dinâmica a serie de dados da pandemia de COVID-19.

2. A interação entre uma população de bactérias e neutrófilos, pode ser descrita pelo seguinte modelo matemático

$$\begin{aligned}\frac{dB}{dt} &= c_p B - \lambda_{nb} NB, \\ \frac{dN}{dt} &= \gamma_n B (cn_{max} - N) - \lambda_{bn} NB - \mu_n N,\end{aligned}$$

onde c_p é a taxa de crescimento da bactéria, λ_{nb} é a taxa de fagocitose dos neutrófilos, γ_n é a permeabilidade dos vasos sanguíneos para a saída dos neutrófilos do sistema circulatório, λ_{bn} é a taxa de apoptose dos neutrófilos e μ_n a taxa de decaimento natural dos neutrófilos. Utilizando os dados disponibilizados no Google Classroom ajuste este modelo com a estratégia abordada.